EKSPLORACJA DANYCH

Statystyki zbiorcze i wizualizacja danych

Paweł Gliwny
Wydział Fizyki i Informatyki Stosowanej, Uniwersytet Łódzki
2026/2027

Od tabeli do wykresu

Na laboratorium mamy już `DataFrame`. Dziś: jak go **opisać** i jak go **pokazać**.

1. Statystyki zbiorcze

Położenie i rozrzut: średnia, mediana, odchylenie, kwartyle. `describe()`.

2. Wizualizacja

Który wykres do jakiego pytania? matplotlib, pandas, seaborn.

3. Format danych

Szeroki vs długi. `melt()`, `pivot()` — i dlaczego seaborn tego wymaga.

Na kolejnym wykładzie złożymy to w całość: eksploracyjna analiza danych (EDA) na prawdziwym zbiorze.

1

Statystyki zbiorcze

Podstawowe statystyki

- **Statystyki zbiorcze** opisują właściwości zbioru danych.
  - Dane nie zawsze mają postać tabeli czy arkusza...
  - ...często dostajemy je właśnie w postaci statystyk zbiorczych.
- **Statystyka opisowa** to pierwszy krok do zrozumienia, co pokazują dane i gdzie potrzebna jest głębsza analiza.
  - Podsumowuje zbiór danych miarami: średnia, mediana, odchylenie standardowe...

Miary położenia i rozrzutu

Miary położenia (tendencji centralnej)
średnia, mediana, moda

Gdzie na osi liczbowej leży typowa wartość?

Miary rozrzutu (zmienności)
rozstęp, wariancja, odchylenie standardowe, IQR

Jak daleko od typowej wartości leżą pozostałe?

Oznaczenia: $x_1, \ldots, x_n$ — obserwacje; $x_{(1)} \leq x_{(2)} \leq \ldots \leq x_{(n)}$ — te same obserwacje **posortowane**.

Średnia, mediana i moda

Średnia (arytmetyczna) — suma wartości podzielona przez ich liczbę. Tyle wnosiłaby każda obserwacja, gdyby wszystkie były równe.

$$\overline{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Mediana — środkowa wartość posortowanych danych.

$$\mathrm{Me} = \begin{cases} x_{((n+1)/2)} & \text{dla } n \text{ nieparzystego} \\ \frac{1}{2}\left(x_{(n/2)} + x_{(n/2+1)}\right) & \text{dla } n \text{ parzystego} \end{cases}$$

Moda (dominanta) — wartość występująca najczęściej.

$$\mathrm{Mo} = \arg\max_{v} \#\{i : x_i = v\}$$

Miary rozrzutu — wzory

Rozstęp — różnica między największą a najmniejszą wartością.

$$R = x_{(n)} - x_{(1)}$$

Wariancja z próby — średni kwadrat odchylenia od średniej (z poprawką $n - 1$).

$$s^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \overline{x})^2$$

Odchylenie standardowe — w tych samych jednostkach co dane.

$$s = \sqrt{s^2}$$

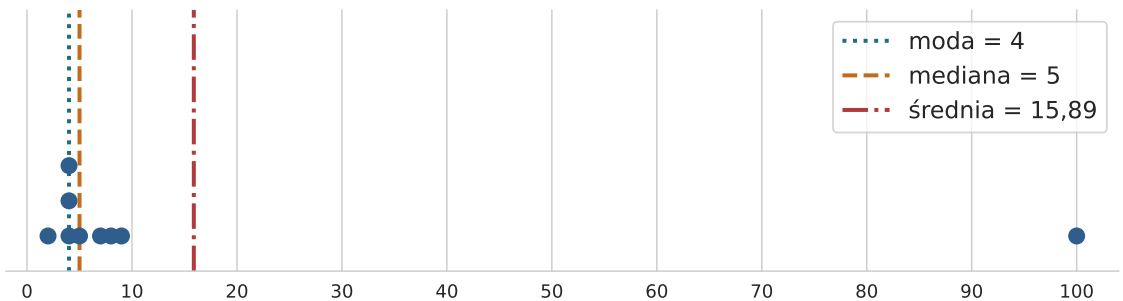
Rozstęp międzykwartylowy — rozrzut „środkowej połowy” danych.

$$\mathrm{IQR} = Q_3 - Q_1$$

Wariancja populacji: $\sigma^2 = \frac{1}{n} \sum (x_i - \mu)^2$ — dzielimy przez n, nie przez $n - 1$.

Przykład

Nasze dane: **[7, 5, 4, 8, 4, 2, 9, 4, 100]**



- Średnia 15,89 nie występuje w danych — i nie jest „typową” wartością.
- Podobnie: LeBron James zdobywa średnio ok. 27 punktów na mecz, choć w żadnym meczu nie zdobył 27,1 punktu.

Przykład c.d. — jedna wartość zmienia wszystko?

Miara	z wartością 100	bez niej
średnia	15,89	5,38
mediana	5	4,5
odch. standardowe	**31,62**	**2,39**
rozstęp	98	7
IQR	**4**	**3,25**

Miary odporne
Mediana i IQR prawie nie reagują na jedną skrajną wartość. Średnia, odchylenie standardowe i rozstęp — bardzo.

```
s = pd.Series([7, 5, 4, 8, 4, 2, 9, 4,
100])
s.std()     # 31.62  (ddof=1 — próba)
np.std(s)   # 29.81  (ddof=0 — populacja!)
```

Częste pomyłki

Mit
Średnia to środkowy punkt danych: połowa liczb leży powyżej średniej, a połowa poniżej.

- **To nieprawda** — tak działa **mediana**.
- Często większość danych leży poniżej (lub powyżej) średniej.
  - Ogromna większość ludzi ma **więcej** palców niż średnia — która wynosi zapewne 9 z kawałkiem.
  - W naszym przykładzie 8 z 9 wartości leży poniżej średniej.

Percentyle

- **Percentyl** P to wartość, dla której:
  - co najmniej $P\%$ obserwacji jest od niej mniejszych lub równych,
  - co najmniej $(100 - P)\%$ obserwacji jest od niej większych lub równych.

Jak obliczyć? (interpolacja liniowa — domyślna w NumPy i pandas)

$$h = (n-1)p, \quad k = \lfloor h \rfloor + 1, \quad Q(p) = x_{(k)} + (h + 1 - k)\left(x_{(k+1)} - x_{(k)}\right)$$

gdzie $p \in [0, 1]$, np. $p = 0.8$ dla 80. percentyla

Percentyle — przykład

80. percentyl zbioru {3, 1, 5, 3, 6, 7, 2, 9}:

- Po sortowaniu: {1, 2, 3, 3, 5, **6**, **7**, 9}, $n = 8$.
- $h = 7 \cdot 0.8 = 5.6$, więc $k = 6$: $Q(0.8) = 6 + 0.6 \cdot (7 - 6) = \mathbf{6.6}$

Uwaga

Istnieje kilka definicji percentyla. Metoda „najbliższej rangi” da tu **7**. W NumPy wybieramy ją parametrem `method` — dlatego różne programy mogą podawać różne wyniki.

```
np.percentile([3, 1, 5, 3, 6, 7, 2, 9], 80)                          # 6.6
np.percentile([3, 1, 5, 3, 6, 7, 2, 9], 80, method="inverted_cdf")   # 7
```

Kwantyle, kwartyle i percentyle

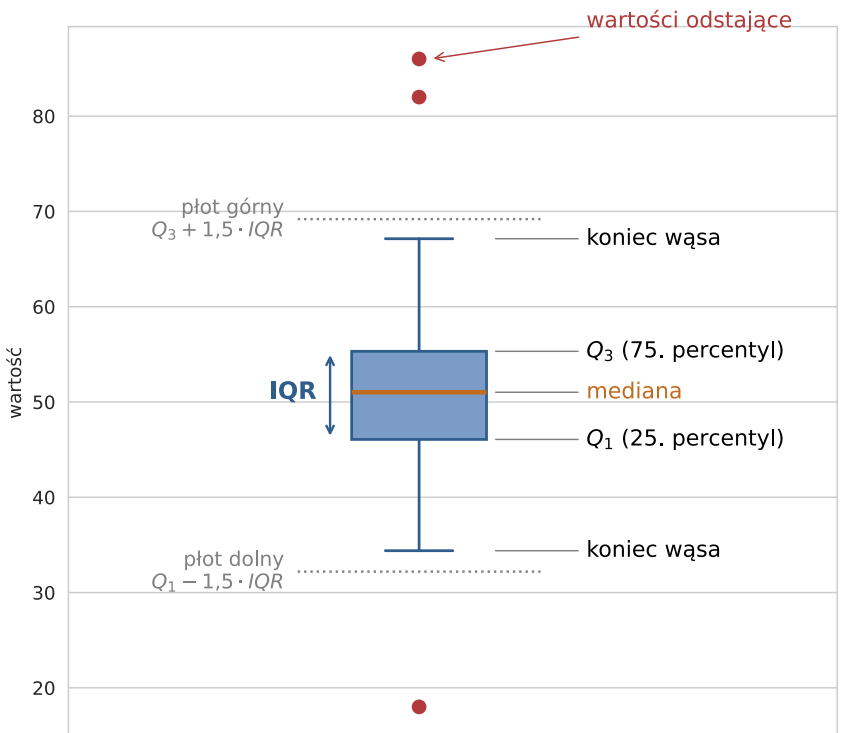
- **Kwantyl** — ogólna nazwa wartości, która dzieli posortowany zbiór w określonej proporcji.
- Kwantyl zapisujemy jako liczbę z przedziału $[0, 1]$:
  - kwantyl 0,25 = 25. percentyl = Q_1,
  - kwantyl 0,50 = 50. percentyl = **mediana** = Q_2,
  - kwantyl 0,75 = 75. percentyl = Q_3,
  - kwantyl 0,80 = 80. percentyl.
- **Kwartyle** — szczególny przypadek: Q_1, Q_2, Q_3.

Wykres pudełkowy (ang. *box plot*)

- **Mediana** = 50. percentyl
- Q_1 = 25. percentyl, Q_3 = 75. percentyl
- Pudełko: od Q_1 do Q_3, jego wysokość to $\mathrm{IQR} = Q_3 - Q_1$
- Płoty (ang. *fences*):

$$Q_1 - 1.5 \cdot \mathrm{IQR}, \quad Q_3 + 1.5 \cdot \mathrm{IQR}$$

- Wąsy kończą się na **najdalszej obserwacji wewnątrz płotów**; punkty poza nimi to kandydaci na wartości odstające.



Statystyki opisowe w pandas: `describe()`

```
miasta["Populacja"].describe()
```

```
count          7.0
mean      765351.0
std       501088.2
min       395513.0
25%       501477.5
50%       643782.0
75%       726075.5
max      1863056.0
```

Wiersz	Miara
`count`	n (bez braków!)
`mean`	średnia $\overline{x}$
`std`	odch. standardowe s (próba, $n-1$)
`25%`, `50%`, `75%`	Q_1, **mediana**, Q_3
`min`, `max`	$x_{(1)}$, $x_{(n)}$

Średnia (765 tys.) > mediana (644 tys.) — ciągnie ją Warszawa.

Pojedyncze statystyki w pandas

```python
s = miasta["Populacja"]

s.mean()                # 765351.0
s.median()              # 643782.0
s.std()                 # 501088.2
s.quantile([0.25, 0.75]) # 501477.5, 726075.5
s.max() - s.min()       # rozstęp: 1467543

miasta["Powierzchnia_km2"].mode()
# 262, 293  ← dwie mody!
```

- Te same nazwy działają na całym `DataFrame` — liczą statystykę **dla każdej kolumny**.
- Moda nie musi być jedna — `mode()` zwraca `Series`.
- Dla danych kategorycznych zamiast średniej: `value_counts()`.

IQR $= Q_3 - Q_1$ = 224 598 — rozpiętość „środkowej połowy” miast.

2

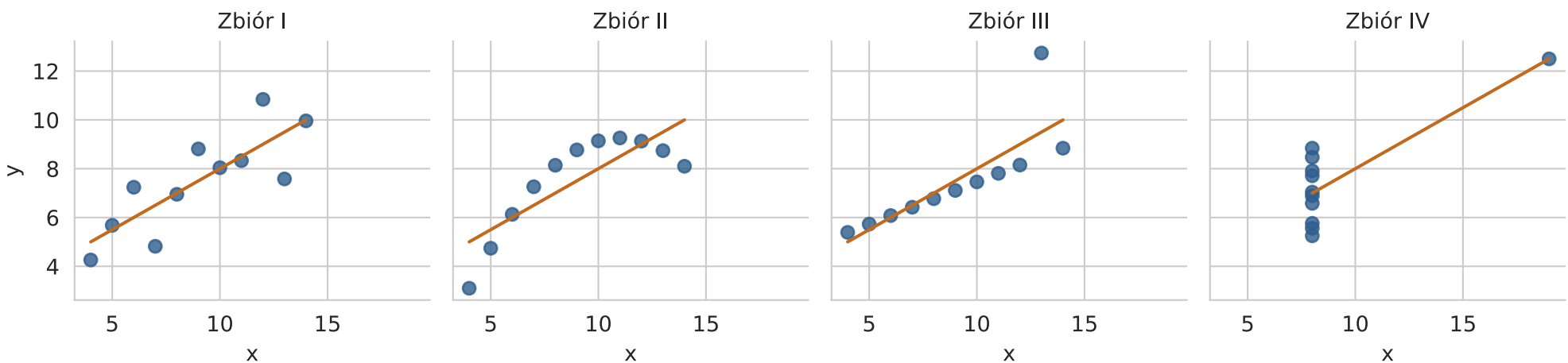
Wizualizacja danych

Wizualizacja danych

- Jeden z pierwszych etapów analizy danych — ułatwia rozumienie i przekazywanie informacji.
- Przedstawiamy dane w formie graficznej: wykresy, schematy, mapy.
- Pozwala wychwytywać **wzorce, trendy, wartości odstające, rozkłady** i wzajemne relacje.
- Pomaga radzić sobie z dużą ilością danych.

Dlaczego nie wystarczą statystyki?

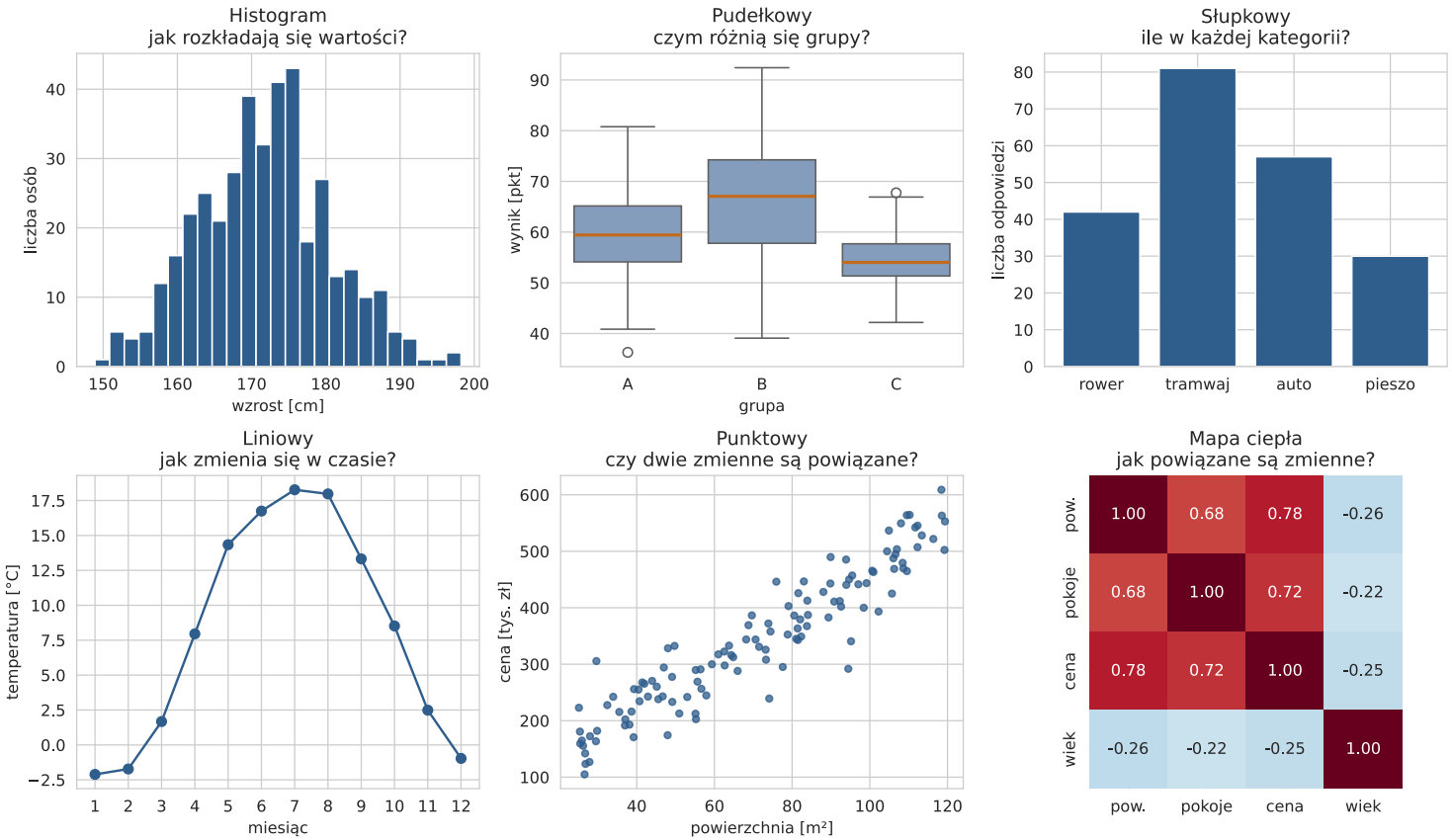
Cztery zbiory po 11 punktów. **Praktycznie identyczne** statystyki: średnia x = 9, średnia y = 7,50, wariancja x = 11, wariancja y ≈ 4,12, korelacja ≈ 0,82, ta sama prosta regresji $y \approx 3 + 0.5x$.



Statystyki zbiorcze **streszczają**, wykres **pokazuje**. Zawsze patrz na jedno i drugie.

Źródło: F. J. Anscombe, *Graphs in Statistical Analysis*, The American Statistician 27(1), 1973

Galeria typów wykresów



Który wykres do jakiego pytania?

Pytanie	Wykres	seaborn
Jak rozkładają się wartości jednej zmiennej liczbowej?	histogram, gęstość (KDE), ECDF	`histplot`, `kdeplot`, `ecdfplot`
Czym różnią się grupy?	pudełkowy, skrzypcowy	`boxplot`, `violinplot`
Ile jest obserwacji w każdej kategorii?	słupkowy	`countplot`, `barplot`
Jak coś zmienia się w czasie?	liniowy	`lineplot`
Czy dwie zmienne liczbowe są powiązane?	punktowy	`scatterplot`
Jak powiązanych jest wiele zmiennych naraz?	mapa ciepła korelacji, macierz wykresów	`heatmap`, `pairplot`

Matplotlib

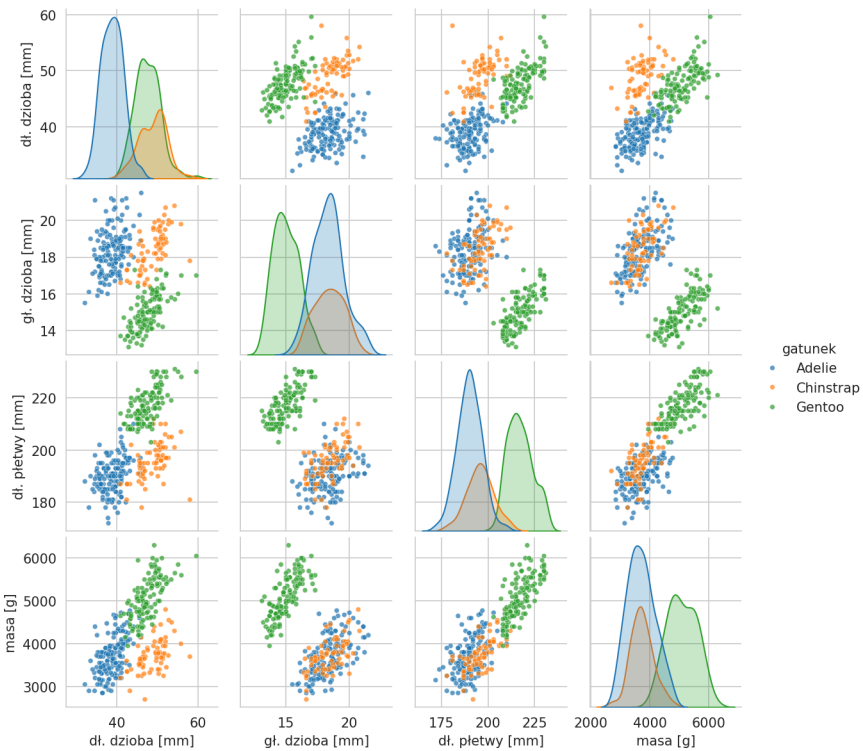
Wszechstronna biblioteka do tworzenia statycznych, animowanych i interaktywnych wizualizacji w Pythonie. *Proste rzeczy są proste, a trudne stają się możliwe.*

- Wykresy o jakości publikacyjnej.
- Pełna kontrola nad stylem i układem.
- Osadzanie w JupyterLab i graficznych interfejsach użytkownika.
- Bogaty ekosystem pakietów opartych na Matplotlib.

Źródło: matplotlib.org

Seaborn

- Biblioteka do wizualizacji danych w Pythonie oparta na matplotlib.
- Wysokopoziomowy interfejs do atrakcyjnych i bogatych w informacje **wykresów statystycznych**.
- Obok: `sns.pairplot` dla danych o pingwinach — rozkłady na przekątnej, zależności poza nią.



Źródło: seaborn.pydata.org dane `penguins`

Ten sam wykres — trzy sposoby

matplotlib

```
fig, ax = plt.subplots()
ax.bar(df["Miasto"],
       df["Populacja"])
ax.set_ylabel("Populacja")
```

Rysujemy „ręcznie” na osiach (`Axes`).

pandas

```
df.plot.bar(x="Miasto",
            y="Populacja")
```

Metoda `DataFrame` — pod spodem i tak matplotlib.

seaborn

```
sns.barplot(data=df,
            x="Miasto",
            y="Populacja",
            hue="Kategoria")
```

Parametry to **nazwy kolumn**.

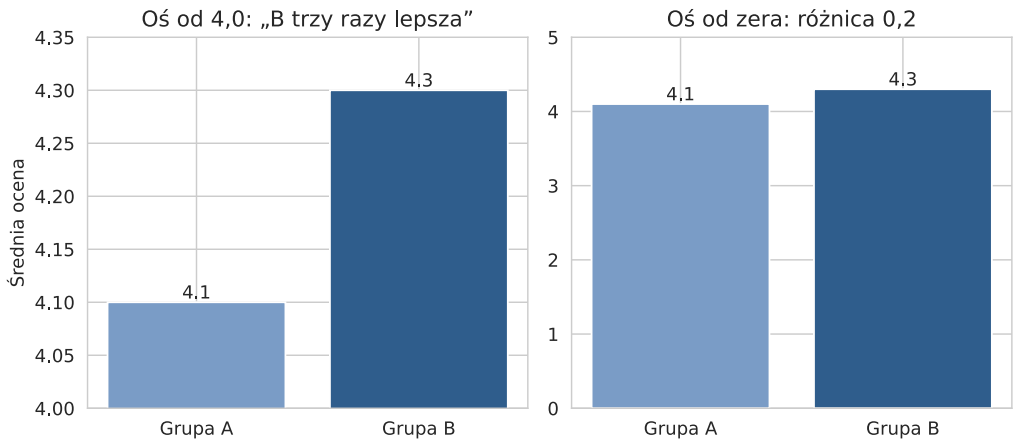
Która biblioteka kiedy?

Biblioteka	Zalety	Wady	Kiedy używać
matplotlib	pełna kontrola, elastyczność	dużo kodu	precyzyjne, publikacyjne wykresy
pandas	szybko, prosto z `DataFrame`	ograniczona stylizacja	szybka eksploracja
seaborn	wykresy statystyczne, `hue`, estetyka	wymaga **formatu długiego**	analiza, raporty, porównania grup

W praktyce: zaczynamy od pandas, do analizy używamy seaborn, a szczegóły (tytuły, osie, rozmiar) poprawiamy w matplotlib.

Typowe błędy na wykresach

- **Ucięta oś Y** na wykresie słupkowym — słupki porównujemy długością, więc oś musi zaczynać się od zera (na wykresie liniowym nie musi).
- **Brak opisów i jednostek** osi.
- **Legenda wpisana ręcznie** w innej kolejności niż dane — wykres „kłamie” po cichu.
- **Za dużo kolorów** albo kolor bez znaczenia.



3

Format danych dla wykresów

Te same dane, dwa kształty

Format szeroki

Miasto	Sty	Kwi	Lip	Paź
Warszawa	−2	9	19	9
Kraków	−1	10	20	9
Gdańsk	0	8	17	10

Wiersz = miasto. Miesiące są **nagłówkami kolumn**.

Format długi

Miasto	Miesiąc	Temp.
Warszawa	Sty	−2
Kraków	Sty	−1
Gdańsk	Sty	0
Warszawa	Kwi	9
…	…	…

Wiersz = **jedna obserwacja**.

Dane uporządkowane (*tidy data*): każda zmienna to kolumna, każda obserwacja to wiersz.

Źródło: H. Wickham, *Tidy Data*, Journal of Statistical Software 59(10), 2014

`melt()` i `pivot()`

```
miesiace = ["Styczeń", "Kwiecień", "Lipiec", "Październik"]

# szeroki → długi:  (6 × 5) → (24 × 3)
dluga = temperatura.melt(id_vars="Miasto", value_vars=miesiace,
                         var_name="Miesiąc", value_name="Temperatura")

# długi → szeroki
szeroka = dluga.pivot(index="Miasto", columns="Miesiąc",
                      values="Temperatura")[miesiace]
```

- `id_vars` — kolumny, które zostają (identyfikują obserwację),
- `value_vars` — kolumny, które „zwijamy” w dwie: nazwę (`var_name`) i wartość (`value_name`).

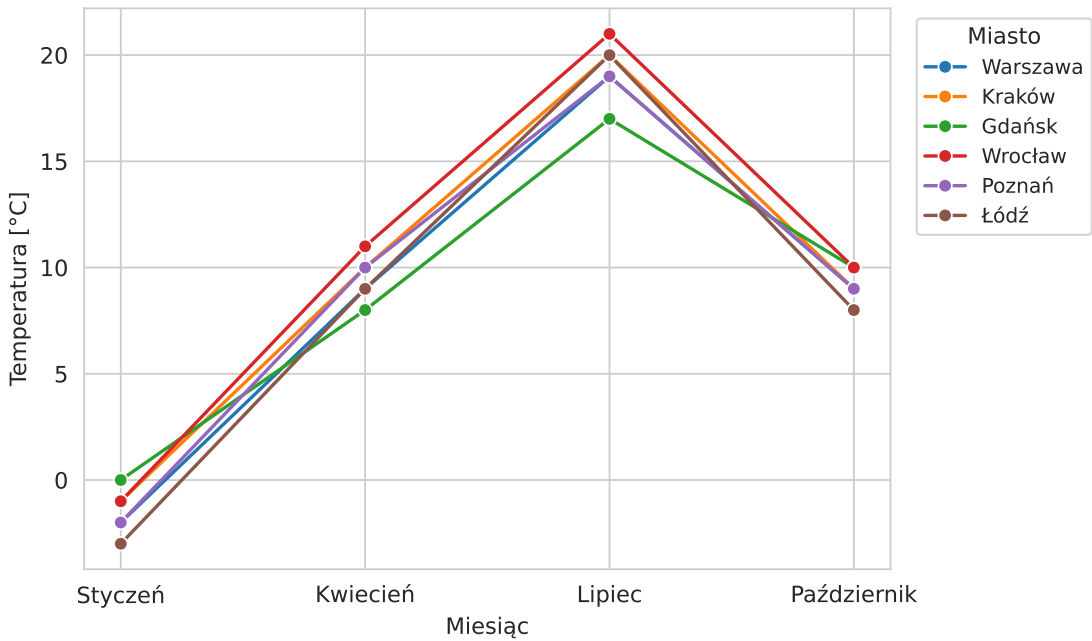
Uwaga

Po `pivot()` kolumny są posortowane **alfabetycznie** — dlatego na końcu `[miesiace]` przywraca kolejność.

Dlaczego seaborn potrzebuje formatu długiego?

- Każdy parametr (`x`, `y`, `hue`, `style`, `size`) to **nazwa kolumny**.
- W formacie szerokim „miesiąc” nie jest kolumną, tylko nagłówkiem — nie da się go podpiąć pod oś.
- pandas `.plot()` działa odwrotnie: każda kolumna to osobna linia → lubi format **szeroki**.

```
sns.lineplot(data=dluga,
             x="Miesiąc",
             y="Temperatura",
             hue="Miasto")
```



Na laboratorium

Część A — pandas

- tworzenie `DataFrame`, `head()`, `info()`, `describe()`,
- sortowanie, wybór kolumn,
- filtrowanie (`&`, `|`, `isin`),
- nowe kolumny, `apply()`.

Część B — wizualizacja

- wykres słupkowy, punktowy, liniowy,
- matplotlib vs pandas vs seaborn,
- `melt()` i `hue`.

Notebook w swoim repozytorium na GitHubie przed kolejnymi zajęciami.

Następny wykład: eksploracyjna analiza danych (EDA) — od Tukeya do studium przypadku cen domów (zbiór Ames).

Źródła

- A. J. Gutman, J. Goldmeier, *Analityk danych. Przewodnik po data science, statystyce i uczeniu maszynowym*, Helion.
- J. W. Tukey, *Exploratory Data Analysis*, Addison-Wesley, 1977.
- F. J. Anscombe, *Graphs in Statistical Analysis*, The American Statistician 27(1), 1973.
- H. Wickham, *Tidy Data*, Journal of Statistical Software 59(10), 2014.
- Dokumentacja: pandas.melt, matplotlib.org, seaborn.pydata.org, numpy.percentile.
- Wykresy: skrypty `wykresy.py` (dane `penguins`) i `wykresy_wyklad2.py`.